Faculdade de Design, Tecnologia e Comunicação
Universidade Europeia

Licenciatura em Engenharia Informática | LEIF005D02 | 26-27
UC | Projeto de Desenvolvimento de Software
Docente | Miguel Jorge Monteiro Roseiro Boavida

RELATÓRIO TÉCNICO – 1ª FASE

PROJETO: PULSECHAT

Márcio Quintas - 20241499
Gianni Lopes - 20240593
Ngweza Van-Dúnem - 20240075

Lisboa, 09 de Outubro de 2026

Resumo

O PulseChat constitui uma plataforma distribuída de mensagens instantâneas concebida para operar sob cenários de indisponibilidade parcial de infraestrutura, garantindo alta disponibilidade, integridade e consistência estrita na troca de mensagens entre utilizadores. A proposta materializa as diretrizes do Trabalho 8 do briefing curricular, cujo foco prioritário é eliminar pontos únicos de falha (*Single Points of Failure* — SPoF) através de filas de mensagens assíncronas, persistência redundante e recuperação orientada a garantias formais.

Palavras-chave

Sistemas distribuídos; mensagens instantâneas; tolerância a falhas; comunicação assíncron; replicação; consistência; cibersegurança; Machine Learning.

Link do repositório no GitHub

O código-fonte, o histórico de alterações, as tarefas e os artefactos de desenvolvimento serão organizados num repositório GitHub.
URL: https://github.com/Ngweza/PulseChat.git

Índice

Resumo 2
Palavras-chave 2
Link do repositório no GitHub 2
1. Proposta Inicial do Projeto 4
2. Enquadramento 4
3. Objetivos 5
4. Público-Alvo 5
5. Funcionalidades Principais 5
6. User Journey 6
7. Casos de Uso 7
8. Arquitetura Inicial 7
9. Componentes do Sistema 8
10. Comunicação Assíncrona 9
11. Persistência e Estrutura da Base de Dados 10
12. Consistência e Ordenação das Mensagens 11
13. Modelo de Faltas 11
14. Garantias de Tolerância a Falhas 13
15. Segurança Informática 13
16. Integridade das Mensagens 14
17. Inteligência Artificial 15
18. Requisitos 15
19. Matriz da Stack Tecnológica 17
20. Metodologia de Desenvolvimento 17
21. Plano de Testes de Tolerância a Falhas 18
22. Conclusão 19
23. Bibliografia 19

1. Proposta Inicial do Projeto

O **PulseChat** é uma proposta de serviço de mensagens instantâneas desenvolvido segundo uma arquitetura distribuída e tolerante a falhas. A plataforma permitirá aos utilizadores comunicar através de conversas privadas e grupos, com suporte para envio e receção de mensagens em tempo real.
O projeto enquadra-se diretamente no Trabalho 8 — Serviço de Mensagens Distribuído, apresentado no briefing da unidade curricular. Este trabalho propõe o desenvolvimento de um sistema de mensagens instantâneas através do qual os utilizadores possam comunicar por clientes web ou móveis, devendo a infraestrutura suportar múltiplos servidores e garantir que as mensagens não sejam perdidas perante a falha de um servidor. O briefing identifica ainda como áreas relevantes as filas de mensagens, comunicação assíncrona, persistência, replicação, consistência e recuperação.
O PulseChat pretende, assim, combinar comunicação em tempo real com uma infraestrutura distribuída, permitindo estudar e demonstrar mecanismos de disponibilidade, persistência, replicação, consistência, segurança e recuperação perante falhas.
Além da componente de Sistemas Distribuídos, a solução integrará mecanismos de Segurança Informática e uma componente de Inteligência Artificial, destinada à deteção automática de spam e comportamentos anómalos.

2. Enquadramento

As aplicações modernas de comunicação instantânea dependem de vários componentes para garantir o funcionamento contínuo do serviço, incluindo servidores de aplicação, mecanismos de comunicação em tempo real, bases de dados e sistemas responsáveis pelo processamento e encaminhamento das mensagens.
Uma arquitetura concentrada numa única instância pode criar pontos únicos de falha. Caso esse servidor fique indisponível, os utilizadores podem deixar de conseguir iniciar sessão, consultar conversas ou enviar e receber mensagens.
O PulseChat propõe uma abordagem distribuída, recorrendo a múltiplas instâncias de serviços e mecanismos de redundância. Desta forma, pretende-se que a falha de um componente não implique necessariamente a indisponibilidade completa da plataforma.
Esta abordagem está alinhada com o briefing, que determina que os projetos identifiquem os pontos únicos de falha e definam mecanismos de redundância e recuperação capazes de manter o serviço disponível perante falhas previamente definidas.
A arquitetura deverá também considerar a redundância das camadas aplicacional, de serviços/backend e de armazenamento, sempre que tecnicamente aplicável.

3. Objetivos

3.1 Objetivo Geral

Desenvolver um serviço de mensagens instantâneas distribuído, seguro e tolerante a falhas, capaz de manter a comunicação entre utilizadores perante falhas controladas de componentes da infraestrutura.

3.2 Objetivos Específicos

- Permitir a criação e gestão de contas de utilizador.
- Implementar autenticação segura.
- Permitir comunicação privada entre dois utilizadores.
- Permitir a criação e utilização de grupos de conversação.
- Permitir o envio e receção de mensagens em tempo real.
- Armazenar as mensagens de forma persistente.
- Garantir a persistência das mensagens perante determinadas falhas.
- Utilizar múltiplas instâncias dos serviços de backend.
- Utilizar comunicação assíncrona através de um sistema de filas.
- Implementar mecanismos de replicação dos dados.
- Implementar mecanismos de deteção e recuperação de falhas.
- Proteger as comunicações entre clientes e servidores.
- Garantir mecanismos de autorização sobre os recursos da plataforma.
- Integrar uma componente de Inteligência Artificial para deteção de spam e comportamentos anómalos.
- Definir um modelo de faltas para a solução.
- Demonstrar experimentalmente a tolerância a falhas através de cenários controlados.

4. Público-Alvo

O PulseChat destina-se a utilizadores que necessitem de uma plataforma de comunicação instantânea para troca de mensagens individuais ou em grupo.
A solução será inicialmente disponibilizada através de uma aplicação móvel.

5. Funcionalidades Principais

5.1 Gestão de Utilizadores

O sistema deverá permitir:

- Criar uma conta;
- Iniciar sessão;
- Terminar sessão;
- Consultar o perfil;
- Alterar dados pessoais;
- Alterar a palavra-passe;

- Bloquear outros utilizadores.

5.2 Gestão de Conversas

O sistema deverá permitir:

- Criar conversas privadas;
- Criar grupos;
- Adicionar participantes;
- Remover participantes;
- Consultar o histórico da conversa;
- Pesquisar mensagens;
- Eliminar mensagens de acordo com as regras definidas pelo sistema.

5.3 Gestão de Mensagens

Cada mensagem deverá possuir informação que permita a sua identificação, associação à conversa e controlo do seu estado.

Os principais dados associados a uma mensagem serão:

- Identificador único;
- Remetente;
- Conversa associada;
- Conteúdo;
- Data e hora;
- Número de sequência;
- Estado de entrega;
- Estado de leitura;
- Informação de integridade.

O ciclo de processamento da mensagem poderá ser representado por:

Enviada → Processada → Entregue → Lida

5.4 Funcionamento Offline

Quando um utilizador estiver temporariamente offline, as mensagens destinadas a esse utilizador deverão permanecer armazenadas.

Após o restabelecimento da ligação, o sistema deverá permitir a sincronização das mensagens pendentes.

Esta funcionalidade será particularmente relevante para demonstrar os mecanismos de persistência e recuperação definidos para o projeto.

6. User Journey

O fluxo principal de utilização do PulseChat será composto pelas seguintes etapas:

1. O utilizador acede à aplicação.
2. Efetua o registo ou inicia sessão.
3. O sistema autentica o utilizador.
4. O utilizador consulta as suas conversas.
5. O utilizador seleciona uma conversa ou cria uma nova.
6. O utilizador escreve e envia uma mensagem.

7. A mensagem é processada pelo backend.
8. A mensagem é encaminhada através do sistema de comunicação assíncrona.
9. A mensagem é persistida.
10. O destinatário recebe a mensagem através da comunicação em tempo real.
11. O estado da mensagem é atualizado.
12. Caso o destinatário esteja offline, a mensagem permanece armazenada para posterior sincronização.

7. Casos de Uso

Os principais casos de uso identificados para a primeira fase são:

UC01 — Registar Utilizador
Permite ao utilizador criar uma nova conta na plataforma.

UC02 — Autenticar Utilizador
Permite ao utilizador iniciar sessão através das suas credenciais.

UC03 — Gerir Perfil
Permite consultar e alterar informações associadas à conta.

UC04 — Criar Conversa
Permite iniciar uma conversa privada com outro utilizador.

UC05 — Criar Grupo
Permite criar uma conversa com vários participantes.

UC06 — Gerir Participantes
Permite adicionar ou remover participantes de uma conversa de grupo.

UC07 — Enviar Mensagem
Permite enviar uma mensagem para uma conversa.

UC08 — Receber Mensagem
Permite receber mensagens em tempo real através do sistema.

UC09 — Consultar Histórico
Permite consultar as mensagens armazenadas numa conversa.

UC10 — Pesquisar Mensagens
Permite procurar mensagens existentes no histórico.

UC11 — Sincronizar Mensagens
Permite recuperar mensagens pendentes após o restabelecimento da ligação.

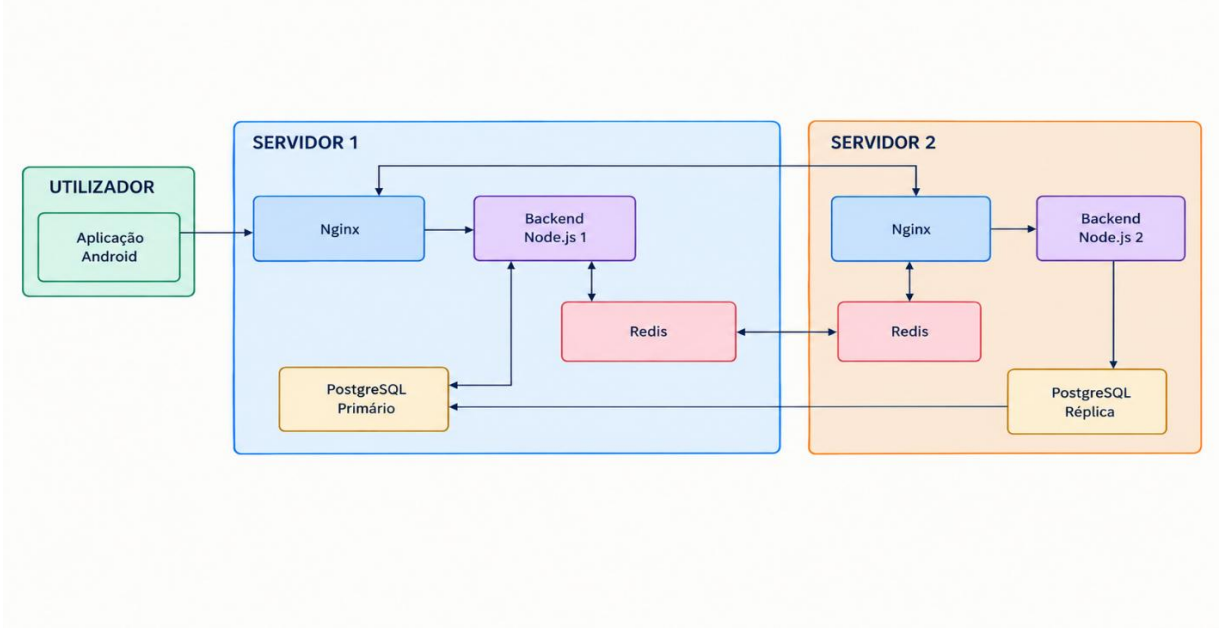
UC12 — Bloquear Utilizador
Permite impedir determinadas interações com outro utilizador.

UC13 — Detetar Comportamento Anómalo
Permite ao sistema analisar padrões de utilização e identificar possíveis situações de spam ou comportamento anómalo.

8. Arquitetura Inicial

A arquitetura inicial proposta para o PulseChat é baseada numa infraestrutura distribuída composta por clientes, um mecanismo de distribuição de pedidos, múltiplas

instâncias de backend, um sistema de mensagens assíncronas, armazenamento persistente e um serviço de Inteligência Artificial.



A utilização de múltiplas instâncias de backend pretende evitar que a falha de uma única instância provoque necessariamente a indisponibilidade do serviço.
O briefing determina que a arquitetura seja documentada e que as principais decisões relacionadas com replicação, consistência, disponibilidade, deteção de falhas e recuperação sejam justificadas.

9. Componentes do Sistema

9.1 Cliente

O cliente será responsável pela interface utilizada pelos consumidores do serviço.
A tecnologia inicialmente proposta é:

- React;
- TypeScript;
- HTML/CSS.

A comunicação com o backend será realizada através de:

- REST API para operações tradicionais;
- WebSocket para comunicação em tempo real.

9.2 Backend

O backend será responsável por:

- Autenticação;
- Autorização;
- Gestão de utilizadores;
- Gestão de conversas;
- Processamento de mensagens;
- Persistência;
- Comunicação com o Message Broker;

- Gestão das ligações WebSocket.

Tecnologias propostas:

- XAMPP.

Serão consideradas pelo menos duas instâncias:

- Backend Node 1;
- Backend Node 2.

9.3 Message Broker

O **RabbitMQ** será utilizado como sistema de Message Broker.
A sua função será permitir comunicação assíncrona entre os componentes responsáveis pelo processamento e entrega das mensagens.

9.4 Base de Dados

Será utilizada uma base de dados PostgreSQL, com uma arquitetura inicialmente prevista de:

- PostgreSQL Primary;
- PostgreSQL Replica.

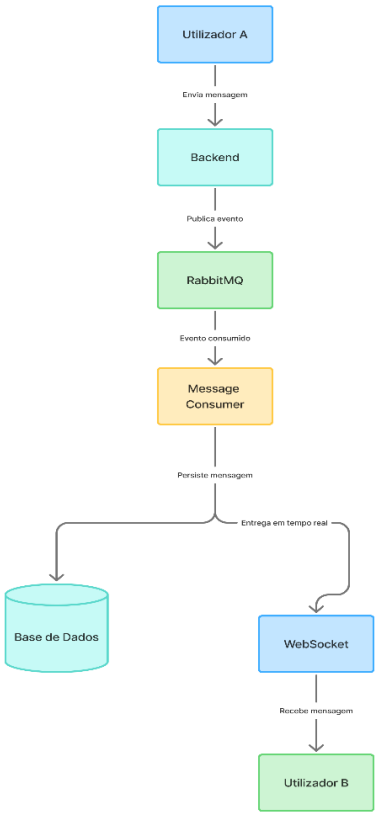
A replicação pretende reduzir a dependência de uma única instância de armazenamento e proporcionar mecanismos de recuperação perante determinadas falhas.

9.5 Serviço de Inteligência Artificial

Será desenvolvido um serviço separado para análise de comportamentos e deteção de possíveis situações de spam ou anomalia.

10. Comunicação Assíncrona

A comunicação assíncrona será um dos elementos centrais da arquitetura do PulseChat.
O fluxo proposto para o envio de uma mensagem será:



A utilização de um Message Broker permite separar o envio da mensagem do seu processamento e entrega.
As mensagens serão colocadas numa fila persistente antes de serem consideradas processadas, permitindo explorar os conceitos de comunicação assíncrona, persistência e recuperação previstos para o Trabalho 8.

11. Persistência e Estrutura da Base de Dados

A estrutura inicial da base de dados será composta pelas seguintes entidades:
USER

- id
- name
- email
- password_hash
- created_at

CONVERSATION

- id
- type
- created_at

CONVERSATION_MEMBER

- conversation_id
- user_id

MESSAGE

- id
- conversation_id
- sender_id
- content
- sequence_number
- status
- created_at

A estrutura será posteriormente detalhada através do respetivo modelo de dados e diagrama da base de dados.

12. Consistência e Ordenação das Mensagens

Um dos problemas a considerar num sistema distribuído de mensagens é a manutenção da ordem correta das mensagens.
Por exemplo, uma situação de concorrência ou processamento distribuído poderia resultar numa sequência incorreta:
Mensagem 1
Mensagem 3
Mensagem 2
quando a ordem correta seria:
Mensagem 1
Mensagem 2
Mensagem 3
Para reduzir este problema, cada conversa terá um mecanismo de ordenação através de um **sequence_number**.
Exemplo:
Conversa = 25

Mensagem A → sequence 101
Mensagem B → sequence 102
Mensagem C → sequence 103
O identificador único da mensagem permitirá também suportar mecanismos de idempotência, evitando que uma mensagem reenviada na sequência de uma falha seja armazenada mais do que uma vez.
A estratégia definitiva de consistência e sincronização será aprofundada durante o desenvolvimento da arquitetura.

13. Modelo de Faltas

O PulseChat terá um modelo de faltas explícito, conforme exigido pelo briefing.
O briefing determina que o primeiro relatório identifique os componentes que podem falhar, os tipos de falha considerados, o número de falhas suportadas, as

funcionalidades que permanecem disponíveis, a deteção e recuperação das falhas, a eventual perda de informação e a consistência entre réplicas.

13.1 Falha do Backend Node 1

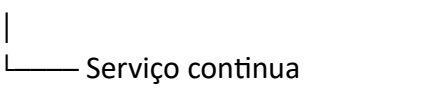
Componente: Backend Node 1
Falha considerada: servidor deixa de responder.
Deteção: health checks.
Resposta: o Load Balancer/Reverse Proxy encaminha novos pedidos para o Backend Node 2.
Impacto esperado: poderá ocorrer uma interrupção temporária de uma ligação WebSocket, mas as mensagens previamente persistidas deverão permanecer disponíveis.
Recuperação: o Backend Node 1 será reiniciado e posteriormente reintegrado na infraestrutura.

13.2 Falha de um Nó do Message Broker

Componente: Message Broker
Falha considerada: indisponibilidade de um nó do sistema de mensagens.
Resposta prevista: utilização de uma configuração redundante do broker, permitindo que os restantes componentes continuem a processar mensagens dentro das garantias definidas pelo projeto.
Recuperação: o componente indisponível será restaurado e a redundância será restabelecida.

13.3 Falha da Réplica da Base de Dados

Componente: PostgreSQL Replica
Falha considerada: indisponibilidade da réplica.



O sistema deverá continuar a funcionar de acordo com a política de disponibilidade definida, enquanto a réplica é recuperada.
Após a recuperação, deverá ser restabelecido o estado redundante da infraestrutura.

13.4 Utilizador Offline

Componente: cliente/utilizador.
Falha considerada: perda temporária da ligação.
Comportamento: as mensagens destinadas ao utilizador não deverão ser eliminadas. As mensagens permanecerão persistidas e pendentes até que o utilizador volte a estabelecer ligação.

13.5 Interrupção da Comunicação

Falha considerada: interrupção temporária da comunicação entre cliente e infraestrutura.

Após o restabelecimento da ligação, o cliente deverá poder sincronizar as mensagens que ficaram pendentes.

14. Garantias de Tolerância a Falhas

Nesta primeira fase são definidas as seguintes garantias iniciais:

- A falha de uma instância de backend não deverá impedir necessariamente o processamento de novos pedidos.
- As mensagens já persistidas deverão permanecer disponíveis perante a falha de um backend.
- As mensagens destinadas a utilizadores offline deverão permanecer armazenadas.
- O sistema deverá possuir mecanismos para recuperação dos componentes que falham.
- Após a recuperação de um componente, deverá ser possível restabelecer a redundância definida na arquitetura.
- A consistência e a integridade das mensagens deverão ser preservadas dentro das garantias definidas pelo sistema.

Estas garantias serão posteriormente validadas através de testes experimentais. O briefing salienta que a simples existência de réplicas não é suficiente: é necessário demonstrar experimentalmente que os componentes restantes conseguem assumir as funções necessárias perante uma falha.

15. Segurança Informática

A segurança será implementada em diferentes camadas da plataforma.

15.1 Proteção das Comunicações

As comunicações deverão utilizar:

- HTTPS;
- WSS (WebSocket Secure).

Desta forma, pretende-se evitar a transmissão de informação em texto simples entre clientes e servidores.

15.2 Autenticação

O modelo de autenticação proposto será:

Email + Password
    |
    ▼
Password Hash
    |

▼

JWT Access Token

+

Refresh Token

As palavras-passe não deverão ser armazenadas diretamente.

15.3 Autorização

Cada pedido deverá ser validado para garantir que o utilizador possui autorização para realizar determinada operação, incluindo:

- Consultar uma conversa;
- Enviar uma mensagem;
- Consultar mensagens;
- Gerir participantes.

15.4 Proteção Contra Abuso

Poderão ser implementados mecanismos como:

- Rate limiting;
- Limitação do tamanho das mensagens;
- Limitação de tentativas de autenticação;
- Bloqueio temporário após excesso de tentativas de login;
- Validação dos dados de entrada;
- Registo de eventos relevantes.
-

16. Integridade das Mensagens

Como mecanismo adicional de integridade, cada mensagem poderá possuir um hash calculado através de SHA-256 sobre o conteúdo e metadados relevantes.

Exemplo conceptual:

Mensagem + Metadata

|

▼

SHA-256

|

▼

Hash

Durante a validação, o hash poderá ser calculado novamente e comparado com o valor armazenado, permitindo identificar alterações nos dados.

A implementação definitiva deste mecanismo será avaliada durante o desenvolvimento do projeto.

17. Inteligência Artificial

O PulseChat integrará uma componente de Inteligência Artificial dedicada à deteção automática de spam e comportamentos anómalos.
Esta componente enquadra-se nos requisitos do briefing, que determina a integração de pelo menos uma componente de IA e apresenta Machine Learning como uma possibilidade para reconhecimento de acessos indesejados ou anomalias.
O módulo poderá analisar características como:

- Frequência de mensagens;
- Número de destinatários;
- Repetição de conteúdos;
- Número de mensagens enviadas num determinado período;
- Padrões anormais de utilização.

A atividade poderá ser classificada como:
NORMAL
SPAM
ANOMALIA
Por exemplo, um comportamento caracterizado pelo envio de um elevado número de mensagens para vários destinatários num período muito reduzido poderá ser identificado pelo serviço como comportamento anómalo.
Dependendo da classificação obtida, o sistema poderá gerar um alerta ou limitar temporariamente determinadas operações.
A técnica de Machine Learning e o modelo definitivo serão selecionados e validados durante o desenvolvimento da componente de IA.

18. Requisitos

18.1. Requisitos Funcionais

RF01 — Registo de utilizador
O sistema deve permitir a criação de contas de utilizador.
RF02 — Autenticação
O sistema deve permitir aos utilizadores iniciar e terminar sessão de forma segura.
RF03 — Gestão de perfil
O sistema deve permitir consultar e alterar os dados do perfil.
RF04 — Criação de conversa privada
O sistema deve permitir criar conversas entre dois utilizadores.
RF05 — Criação de grupos
O sistema deve permitir criar grupos de conversação.
RF06 — Gestão de participantes
O sistema deve permitir adicionar e remover participantes de grupos.
RF07 — Envio de mensagens
O sistema deve permitir enviar mensagens para uma conversa.

RF08 — Receção de mensagens
O sistema deve permitir receber mensagens em tempo real.
RF09 — Persistência de mensagens
O sistema deve armazenar as mensagens de forma persistente.
RF10 — Consulta de histórico
O sistema deve permitir consultar mensagens anteriores.
RF11 — Pesquisa de mensagens
O sistema deve permitir pesquisar mensagens existentes.
RF12 — Sincronização após ligação
O sistema deve permitir recuperar mensagens pendentes após o restabelecimento da ligação.
RF13 — Bloqueio de utilizadores
O sistema deve permitir bloquear outros utilizadores.
RF14 — Deteção de spam e anomalias
O sistema deve analisar comportamentos e identificar possíveis situações de spam ou anomalia.
RF15 — Gestão de estados das mensagens
O sistema deve permitir controlar o estado das mensagens entre enviadas, processadas, entregues e lidas.

18.2. Requisitos Não Funcionais
RNF01 — Disponibilidade
O sistema deverá manter o serviço disponível perante as falhas previstas no modelo de faltas.
RNF02 — Tolerância a falhas
A arquitetura deverá possuir mecanismos de redundância e recuperação.
RNF03 — Persistência
As mensagens deverão ser armazenadas de forma persistente.
RNF04 — Segurança das comunicações
As comunicações deverão utilizar mecanismos seguros como HTTPS e WSS.
RNF05 — Segurança das credenciais
As palavras-passe não deverão ser armazenadas em texto simples.
RNF06 — Escalabilidade
A arquitetura deverá permitir múltiplas instâncias dos serviços de backend.
RNF07 — Consistência
O sistema deverá preservar a ordem e integridade das mensagens de acordo com as garantias definidas.
RNF08 — Integridade
O sistema deverá possuir mecanismos para verificar a integridade dos dados.
RNF09 — Modularidade
Os diferentes componentes deverão ser desenvolvidos de forma modular.
RNF10 — Monitorização e deteção de falhas
Os componentes críticos deverão possuir mecanismos que permitam detetar situações de indisponibilidade.

RNF11 — Versionamento
O desenvolvimento deverá ser acompanhado através do GitHub, conforme definido no briefing.

19. Matriz da Stack Tecnológica

Em consonância com as ferramentas recomendadas para o semestre e o ecossistema moderno de desenvolvimento de sistemas distribuídos:

Camada do Sistema	**Tecnologia Selecionada**	**Justificação Arquitetural**
Frontend	Android Studio + Kotlin	Interface declarativa, tipagem estrita e gestão robusta de estado reativo em sockets.
Backend	XAMPP	Arquitetura empresarial modular baseada em injeção de dependências e suporte nativo a microserviços e WebSockets.
Base de Dados	PostgreSQL	RDBMS transacional com suporte a integridade referencial estrita e replicação por streaming.
Mecanismo de IA	Python (Scikit-Learn)	Ecossistema maduro para modelação rápida, treino e inferência de Machine Learning.
Infraestrutura e Proxy	Docker, Docker Compose	Isolamento e orquestração de nós para simulação fidedigna de falhas parciais.
Auditoria e Controlo	Git + GitHub	Rastreabilidade de código, submissão contínua de artefactos e suporte auditável.

20. Metodologia de Desenvolvimento

O desenvolvimento do PulseChat seguirá uma metodologia ágil, permitindo dividir o projeto em diferentes ciclos de desenvolvimento.
A metodologia proposta contempla:
Sprint 1 — Análise e Requisitos

Análise do problema, definição dos requisitos e identificação dos principais casos de uso.

Sprint 2 — Modelação e Arquitetura

Desenvolvimento dos modelos UML, arquitetura distribuída e modelo inicial da base de dados.

Sprint 3 — Backend e Autenticação

Implementação da estrutura inicial do backend e dos mecanismos de autenticação.

Sprint 4 — Conversas e Mensagens

Implementação das conversas privadas, grupos e gestão das mensagens.

Sprint 5 — WebSocket e RabbitMQ

Implementação da comunicação em tempo real e comunicação assíncrona.

Sprint 6 — Replicação e Tolerância a Falhas

Implementação e validação dos mecanismos de redundância, replicação e recuperação.

Sprint 7 — Segurança

Implementação dos mecanismos de segurança da plataforma.

Sprint 8 — Inteligência Artificial

Desenvolvimento e integração do módulo de deteção de spam e anomalias.

Sprint 9 — Testes e Integração

Realização de testes funcionais, integração e testes de falhas.

Sprint 10 — Demonstração e Documentação

Preparação da demonstração final e consolidação da documentação.

A utilização de uma metodologia ágil está de acordo com o processo de desenvolvimento definido no briefing.

21. Plano de Testes de Tolerância a Falhas

A tolerância a falhas deverá ser demonstrada experimentalmente através de diferentes cenários.

Teste A — Falha do Backend

1. Colocar o Backend Node 1 em funcionamento.
2. Desligar ou interromper o Backend Node 1.
3. Verificar a deteção da falha.
4. Confirmar que o Backend Node 2 continua disponível.
5. Enviar novos pedidos.
6. Verificar se as mensagens continuam a ser processadas.
7. Recuperar o Backend Node 1.
8. Confirmar o restabelecimento da redundância.

Teste B — Falha do Message Broker

1. Interromper um nó do sistema de mensagens.
2. Verificar o comportamento dos restantes componentes.
3. Confirmar se as mensagens continuam a ser processadas dentro das garantias definidas.
4. Recuperar o componente.
5. Confirmar o restabelecimento da infraestrutura.

Teste C — Falha da Réplica da Base de Dados

1. Interromper a réplica.
2. Verificar a disponibilidade do serviço.
3. Confirmar o comportamento do PostgreSQL Primary.
4. Recuperar a réplica.
5. Restabelecer a redundância.

Teste D — Utilizador Offline

1. Desligar a ligação do utilizador.
2. Enviar uma mensagem para esse utilizador.
3. Confirmar que a mensagem permanece persistida.
4. Restabelecer a ligação.
5. Confirmar a sincronização da mensagem pendente.

Teste E — Recuperação

1. Provocar uma falha num componente.
2. Verificar a resposta da infraestrutura.
3. Recuperar o componente.
4. Confirmar a sua reintegração.
5. Verificar se o sistema regressa ao estado redundante.

Estes cenários estão alinhados com o requisito do briefing de demonstrar falhas de serviços, servidores, réplicas de base de dados, comunicações e recuperação dos componentes.

22. Conclusão

O PulseChat reúne todos os pilares essenciais de um sistema informático de grau de engenharia: combina desacoplamento assíncrono com escalabilidade horizontal, garante a integridade e ordenação das comunicações e adota mitigação ativa de falhas nas camadas aplicacional, de mensagens e de dados. A integração do módulo de IA e a abordagem estrita de segurança informática consolidam a resposta às exigências pedagógicas e técnicas do briefing do projeto.

23. Bibliografia

1. RabbitMQ. (s.d.). Consumer Acknowledgements and Publisher Confirms.
2. PostgreSQL Global Development Group. (s.d.).
3. OWASP Foundation. (s.d.). WebSocket Security Cheat Sheet.
OWASP Cheat Sheet Series
4. scikit-learn developers. (s.d.). scikit-learn: Machine Learning in Python — User Guide.